



CHUYÊN ĐỀ TỐT NGHIỆP CÔNG NGHỆ PHẦN MỀM

TS. Đặng Thành Trung
Bộ môn Công nghệ phần mềm
Email: trungdt@hnue.edu.vn

Giới
thiệu

Quy
trình

Yêu cầu

Thiết kế

Cài đặt

ĐBCL

Vận
hành

CHƯƠNG 6

KIỂM THỬ VÀ ĐẢM BẢO CHẤT LƯỢNG PHẦN MỀM

Nội Dung

- **Tổng quan về kiểm thử phần mềm**
- Kiểm thử tĩnh
- Kiểm thử động
- Quản lý kiểm thử
- Công cụ kiểm thử

Kiểm Thử

- **Kiểm thử phần mềm** (Theo IEEE) là quá trình khảo sát hay đánh giá một hệ thống hoặc thành phần hệ thống bằng các phương tiện thủ công hoặc tự động để xác minh rằng nó có thỏa mãn các yêu cầu được đặc tả.

Mục Tiêu Của Kiểm Thử

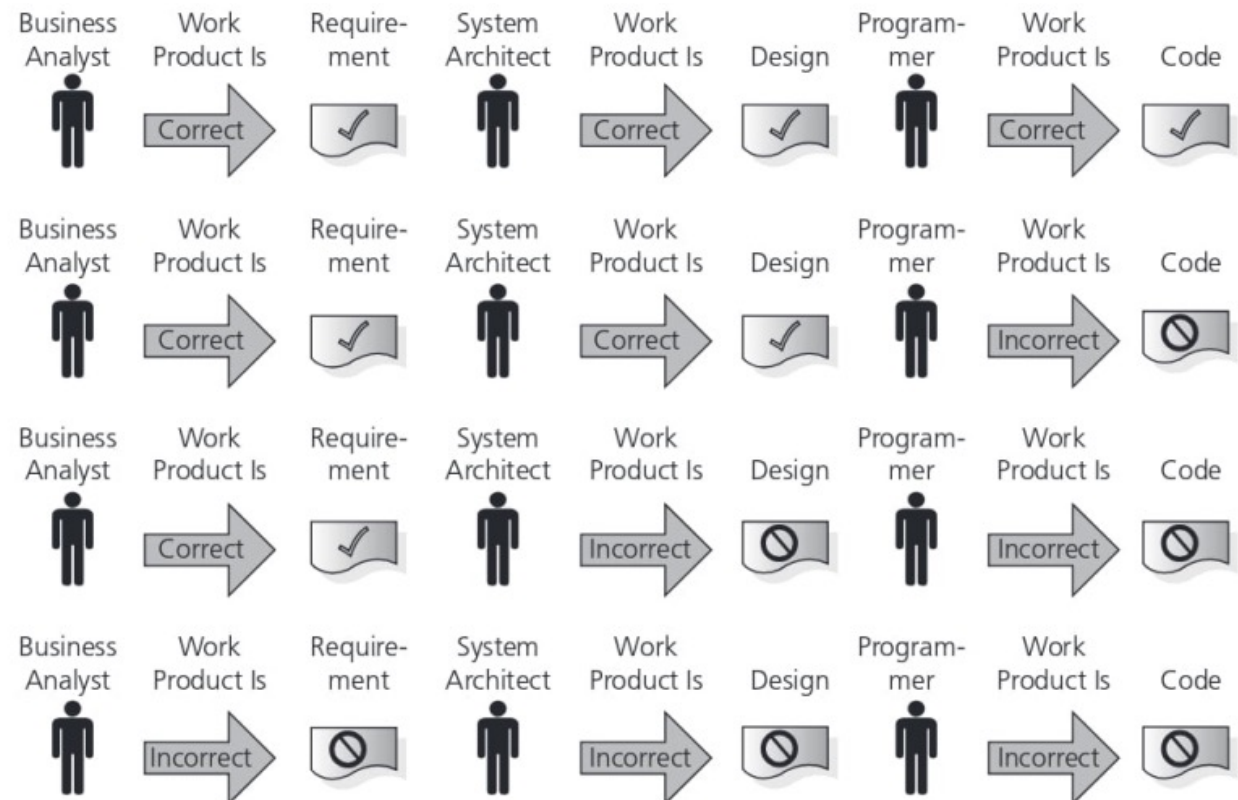
- Tìm khiếm khuyết.
- Đạt được sự tự tin về chất lượng.
- Cung cấp thông tin.
- Ngăn cản khiếm khuyết.
- Để đánh giá các sản phẩm công việc.
- Để kiểm chứng các yêu cầu đã đặc tả có được đáp ứng hay không.
- Để giảm mức độ rủi ro.
- Để tuân thủ các yêu cầu về hợp đồng, pháp lý, hay các tiêu chuẩn, quy định.

Nguyên Nhân Lỗi Phần Mềm

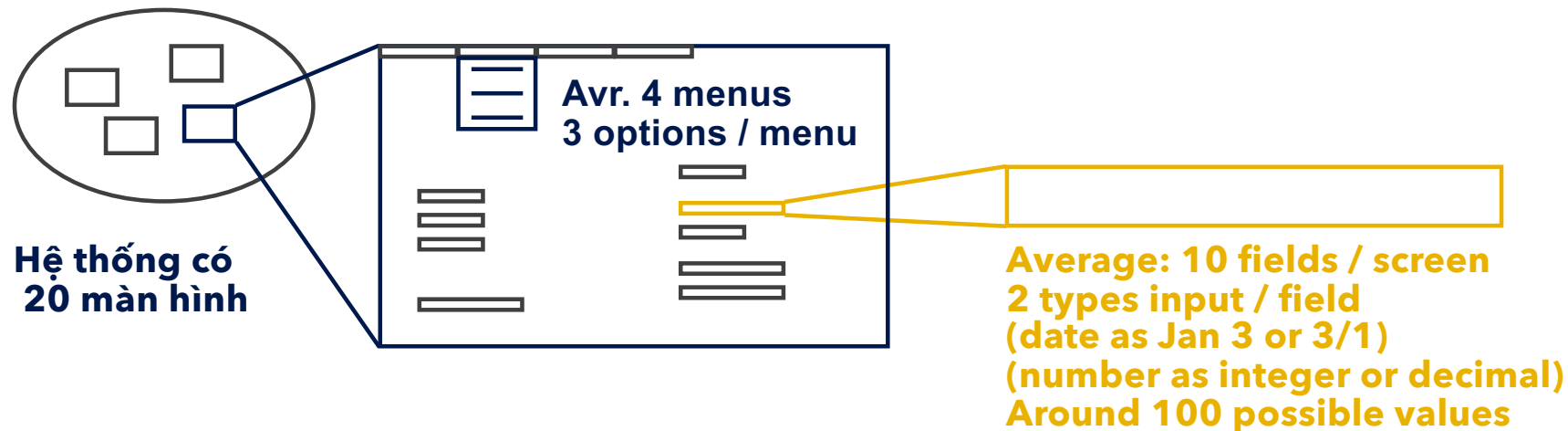
- Tất cả mọi người đều mắc lỗi.
- Sức ép thời gian của dự án phần mềm.
- Độ phức tạp của công việc, kiến trúc, thiết kế hay mã nguồn.
- Hiểu nhầm giữa các thành viên tham gia dự án.
- Hiểu nhầm liên quan tới giao tiếp các thành phần bên trong hệ thống hoặc giữa hệ thống với các hệ thống khác.
- Độ phức tạp của công nghệ được sử dụng.
- Các thành viên đội dự án không đủ kinh nghiệm hoặc không được đào tạo thích hợp.

Các Kịch Bản Phổ Biến

- Theo Capers Jones 2008
 - Requirement: 20%
 - Design: 25%
 - Code: 55%
- Theo một số chuyên gia khác:
 - Requirement + design (~75%)



Tại Sao Không Kiểm Thử Mọi Thứ



Total for 'exhaustive' testing:

$$20 \times 4 \times 3 \times 10 \times 2 \times 100 = 480,000 \text{ tests}$$

If 1 second per test, 8000 mins, 133 hrs, 17.7 days
(not counting finger trouble, faults or retest)

10 secs = 34 wks, 1 min = 4 yrs, 10 min = 40 yrs

Kiểm Thử Bao Nhiều?

- Phụ thuộc vào rủi ro (RISK).
- Sử dụng rủi ro (RISK) để quyết định:
- Cái gì kiểm thử trước;
- Cái gì kiểm thử nhiều nhất;
- Làm thế nào để kiểm tra kỹ lưỡng từng mục;

Sử dụng rủi ro để phân bổ thời gian phù hợp cho việc kiểm thử bằng cách phân quyền ưu tiên kiểm thử ...

Kiểm Thử Và Chất Lượng

- Kiểm thử để đo lường chất lượng phần mềm.
- Kiểm thử có thể tìm ra sai sót, khi sai sót được loại bỏ, chất lượng phần mềm được cải thiện.
- Kiểm thử kiểm tra cái gì?
 - Chức năng hệ thống, tính đúng đắn các thao tác.
 - Các thuộc tính phi chức năng: độ tin cậy, khả năng sử dụng được, khả năng bảo trì, hiệu năng,....

Quy Trình Kiểm Thử

Lập kế hoạch kiểm thử chi tiết

Đặc tả

Thực thi

Ghi chép

**Kiểm tra
hoàn tất**

Đặc Tả Kiểm Thử

**Task 1: Xác
định điều
kiện kiểm thử**

**Task 2: Thiết
kế các test
case**

**Task 3: Xây
dựng các test
case**

Thực Thi Kiểm Thử

- Cài đặt môi trường khai thác khai thác kiểm thử: test driver, simulator, test tools,...
- Thực hiện các test case đã được chỉ định.
 - Các test case quan trọng thực thi trước.
 - Sẽ không thực hiện hết tất cả các test case nếu:
 - Chỉ kiểm thử để sửa lỗi;
 - Có quá nhiều lỗi được tìm thấy bởi các trường hợp kiểm thử đơn giản;
 - Sức ép thời gian.
 - Ghi lại test log.
 - Tái sử dụng test case.
 - Có thể thực hiện các ca kiểm thử thủ công hay sử dụng công cụ kiểm thử tự động.

Ghi Chép Kiểm Thử

- Đánh giá kết quả kiểm thử dựa trên tiêu chí kết thúc trong test plan.
- Ghi chép mức độ bao phủ đạt được (ứng với các độ đo được đặc tả như là các tiêu chí hoàn tất kiểm thử).
- Sau khi mỗi sai sót được điều chỉnh, lặp lại các hoạt động kiểm thử được yêu cầu (thực thi, thiết kế, lên kế hoạch).
- Lập báo cáo kết quả thực hiện kiểm thử.

Kiểm Tra Hoàn Tất Kiểm Thử

- Xác định kết thúc hay tiếp tục kiểm thử dựa trên tiêu chí hoàn tất kiểm thử xác định trong kế hoạch kiểm thử.
- Nếu chưa đạt được, cần lặp lại các hoạt động kiểm thử. Vd thiết kế thêm test case.

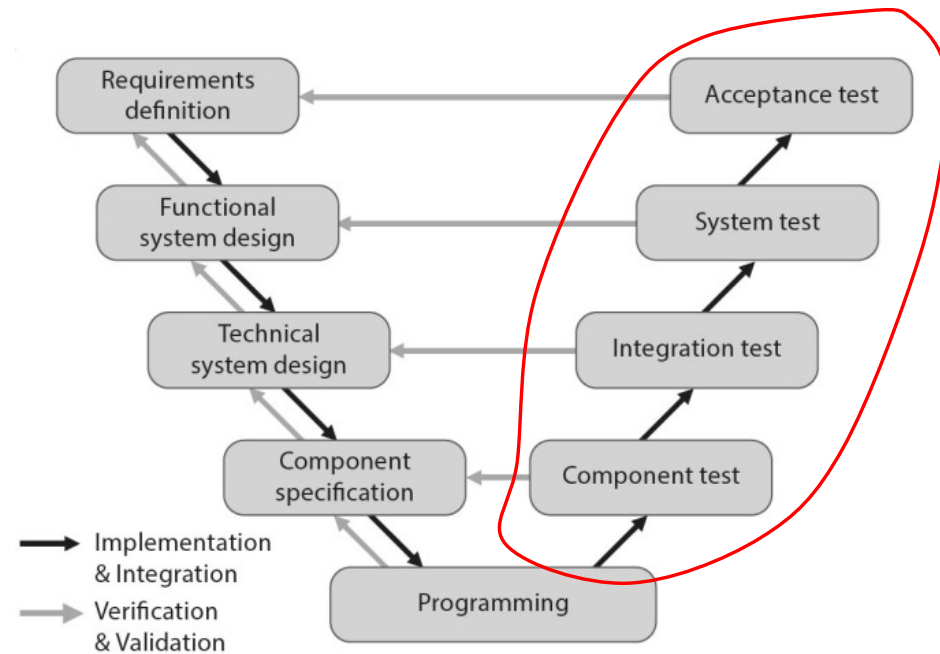


Sản Phẩm Kiểm Thử

Lập kế hoạch	• Test plan
Giám sát	• Test reports
Phân tích	• Test conditions
Thiết kế	• Test cases (high level), test data, test environments, tools
Cài đặt	• Test cases (low level): produces, execution schedule, oracles
Thực thi	• Test case status (ready to run, passed, failed, blocked, skipped)
Đánh giá và báo cáo	• Test summary reports, change requests,...
Kết thúc	• Các quyết định kiểm thử, rút kinh nghiệm

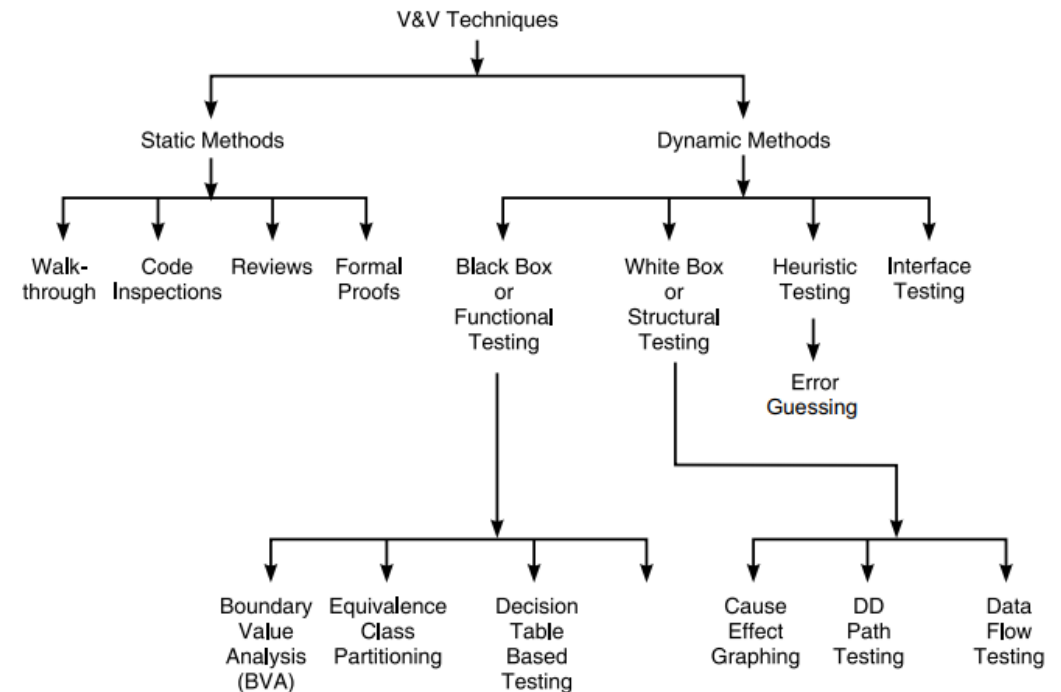
Mức Độ Kiểm Thử

- Kiểm thử thành phần
- Kiểm thử tích hợp
- Kiểm thử hệ thống
- Kiểm thử chấp nhận



Kiểm Thử Tĩnh Và Tiến Trình Kiểm Thử

- **Kiểm thử động (Dynamic testing)**
 - Thực thi hệ thống hoặc thành phần của hệ thống dưới các điều kiện kiểm thử để phát hiện các sai sót bên trong các sản phẩm công việc (work products).
- **Kiểm thử tĩnh (Static testing)**
 - Phân tích và đánh giá để phát hiện sai sót bên trong các work products mà không cần thực thi mã nguồn.



Nội Dung

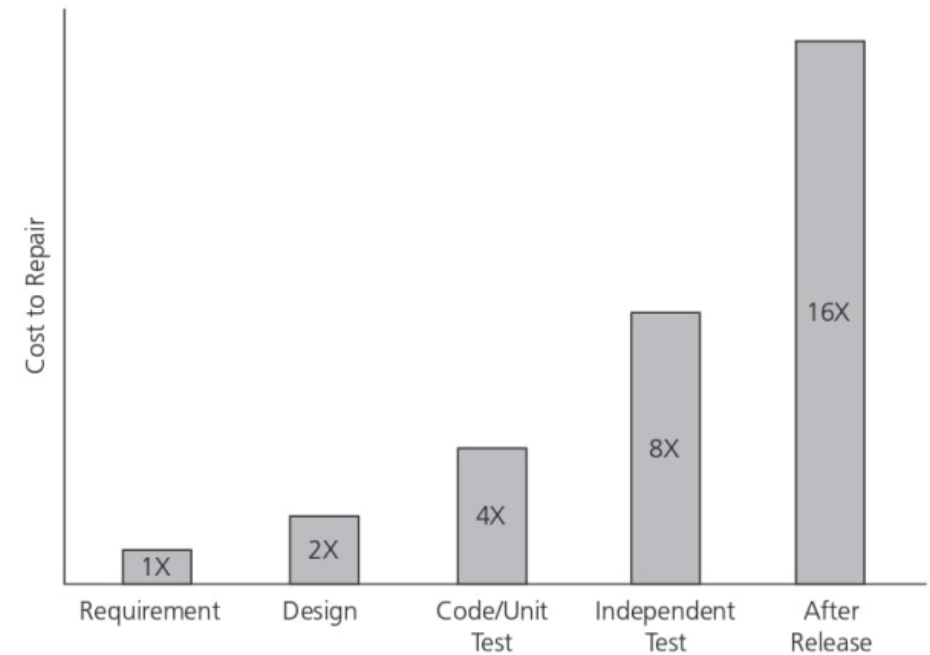
- Tổng quan về kiểm thử phần mềm
- **Kiểm thử tĩnh**
- Kiểm thử động
- Quản lý kiểm thử
- Công cụ kiểm thử

Kiểm Thử Tĩnh

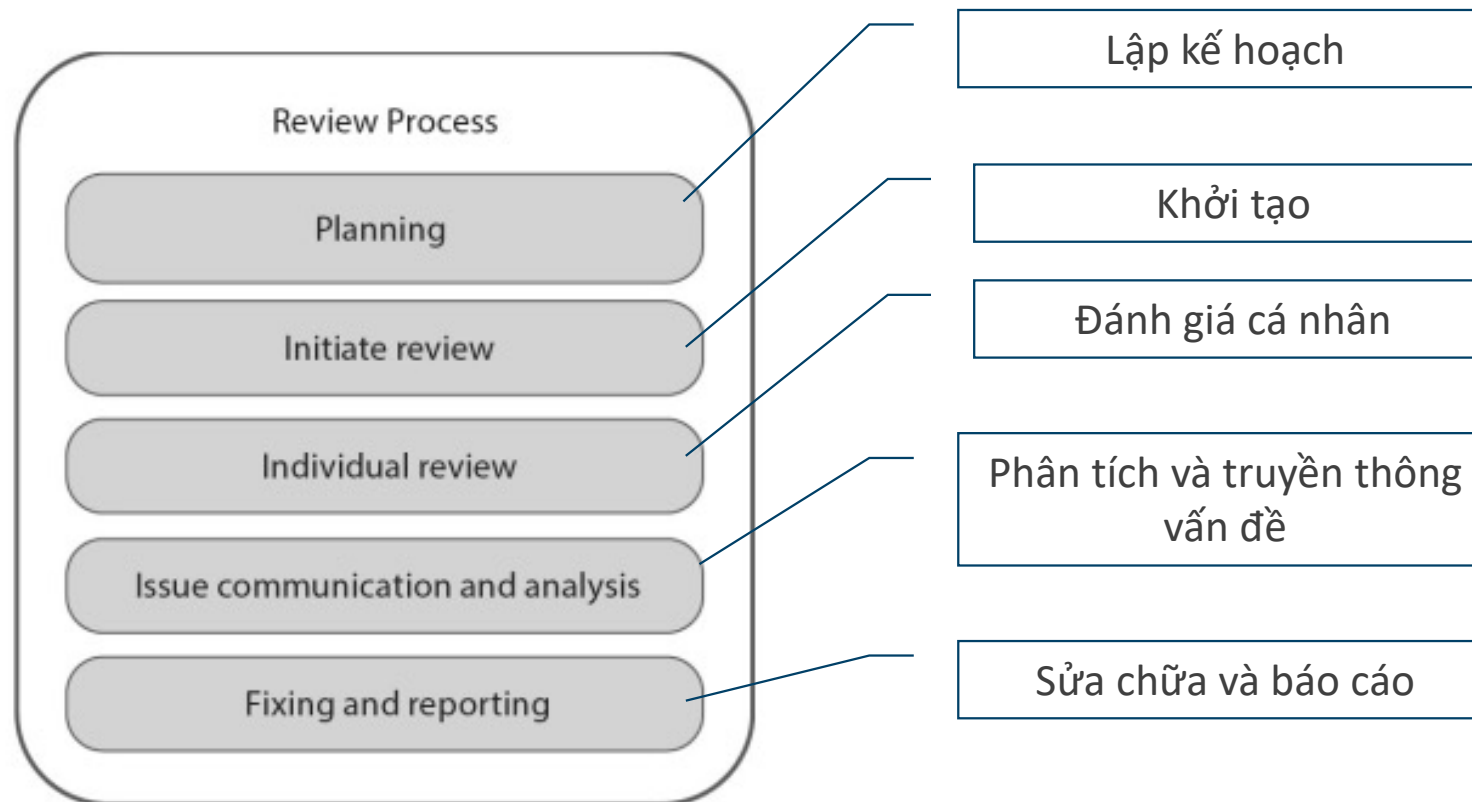
- Bất cứ work products nào của tiến trình phần mềm cũng có thể sử dụng kiểm thử tĩnh:
 - Tài liệu đặc tả yêu cầu.
 - Các thiết kế giải pháp.
 - Mã nguồn.
 - Tài liệu kiểm thử.
 - Hướng dẫn sử dụng.
 - Hợp đồng.
 - ...

Lợi Ích Kiểm Thử Tĩnh

- Có thể tiến hành sớm trong vòng đời phát triển phần mềm và đưa ra các phản hồi sớm các vấn đề về chất lượng.
- Phát hiện lỗi sớm giúp giảm chi phí sửa chữa, chi phí kiểm thử, chi phí rủi ro trong vòng đời phát triển.
- Giảm thời gian phát triển.
- Cải tiến năng suất đội phát triển.
- Giảm mức độ lỗi.
- Cải thiện quan hệ với khách hàng.



Đánh Giá (Review)



Các Loại Đánh Giá

- Informal reviews

- Mục đích chính: phát hiện sai sót tiềm năng.
- Mục đích khác: đưa ra các ý tưởng và giải pháp mới, giải quyết nhanh những sai sót nhỏ.
- Thường không cần tài liệu hóa.
- Không có review meeting.
- Có thể sử dụng hoặc không sử dụng checklist.
- Rất hay được sử dụng trong phát triển Agile.

- Formal reviews

- Phát hiện các sai sót dựa trên các quy định đảm bảo chất lượng của dự án.
- Yêu cầu tài liệu hóa quy trình review.
- Có cuộc họp review và thường có bổ nhiệm nhiều vai trò thành viên liên quan.
- Có review meeting.
- Yêu cầu sử dụng checklist.
- Có tiêu chí kết thúc và báo cáo kết quả review.

Các Loại Đánh Giá

- Informal review - review không chính thức (buddy check, pairing, pair review).
- Hướng dẫn - Walkthrough.
- Peer review/technical review - review ngang hàng/ review kỹ thuật (là một loại formal review).
- Inspection - Thanh tra (the most formal review type).

Đánh Giá Tài Liệu

- Sử dụng các tài liệu mẫu chuẩn

IEEE software life cycle

SQA – Software quality assurance IEEE 730

SCM – Software configuration management IEEE 828

STD – Software test documentation IEEE 829

SRS – Software requirements specification IEEE 830

V&V – Software verification and validation IEEE 1012

SDD – Software design description IEEE 1016

SPM – Software project management IEEE 1058

SUD – Software user documentation IEEE 1063

V • T • E

Đánh Giá Mã Nguồn

- Tuân thủ các quy tắc viết mã của tổ chức
- Tuân thủ các chuẩn viết mã nguồn của cộng đồng
- Phương thức thực hiện:
 - Thủ công,
 - Thân tích mã nguồn tự động sử dụng công cụ (vd SonarQube, PMD, Eslint,...)

PHP: <https://github.com/php-fig/fig-standards>

Swift: <https://google.github.io/swift/>

Html, CSS:

<https://google.github.io/styleguide/htmlcssguide.html>

Javascript:

<https://google.github.io/styleguide/jsguide.html>

C++:

<https://google.github.io/styleguide/cppguide.html>

<https://gist.github.com/lefticus/10191322>

C#: <https://google.github.io/styleguide/csharp-style.html>

Java:

<https://google.github.io/styleguide/javaguide.html>

Python:

<https://google.github.io/styleguide/pyguide.html>

Nội Dung

- Tổng quan về kiểm thử phần mềm
- Kiểm thử tĩnh
- **Kiểm thử động**
- Quản lý kiểm thử
- Công cụ kiểm thử

Kiểm Thử Hộp Đen

1	Phân lớp tương đương
2	Phân tích giá trị biên
3	Đồ thị nhân quả - Bảng quyết định
4	Đoán lỗi
5	Kiểm thử dựa trên mô hình

Kỹ Thuật Phân Lớp Tương Đương

- Các test case kích hoạt thành phần phần mềm (TPPM) thực hiện cùng một hành vi nhóm vào 1 nhóm (họ) → gọi là 1 lớp tương đương.
- Mỗi nhóm chỉ định 1 test case đại diện và dùng test case này để kiểm thử thành phần phần mềm.
- Nếu test case trong lớp tương đương gây lỗi thành phần phần mềm thì các test case khác trong nhóm kỳ vọng cũng sẽ gây ra lỗi như vậy và ngược lại.

Phân Tích Giá Trị Biên

- **Nguyên lý:** Các chương trình có thể coi là một hàm (toán học).
 - Các đầu vào của chương trình là miền xác định của hàm.
 - Các đầu ra là miền giá trị của hàm.
- **Mục tiêu:** Sử dụng kiến thức về hàm để xác định các ca kiểm thử. Trước kia chủ yếu tập trung vào miền xác định, nhưng nay đã dựa trên cả miền giá trị của hàm để xác định ca kiểm thử.
- **Nội dung:** Tập trung phân tích các giá trị biên của miền dữ liệu để xây dựng dữ liệu kiểm thử.
- BVA là kỹ thuật kiểm thử hàm phổ biến nhất.

Chọn Dữ Liệu Kiểm Thử

- Nguyên tắc chung là kiểm thử các dữ liệu: kiểm thử giá trị biên và giá trị cần kề với biên của dữ liệu.
- Nếu dữ liệu vào thuộc một khoảng, chọn:
 - 2 giá trị biên.
 - 4 giá trị = 2 giá trị biên $\pm$ sai số nhỏ nhất.
- Nếu giá trị thuộc vào danh sách các giá trị, chọn:
 - Phần tử thứ nhất, phần tử thứ 2, phần tử kế cuối và phần tử cuối.
- Nếu dữ liệu vào là điều kiện ràng buộc số giá trị, chọn:
 - số giá trị tối thiểu, số giá trị tối đa và một số giá trị không hợp lệ.
- Tự vận dụng khả năng và thực tế để chọn các giá trị cần kiểm thử.

Chọn Ví Dụ Kiểm Thử

Customer Name

Account number

Loan amount requested

☒ **Term of loan**

☐ **Monthly repayment**

Term:

Repayment:

Interest rate:

Total paid back:

2-64 chars.

6 digits, 1st non-zero

£500 to £9000

1 to 30 years

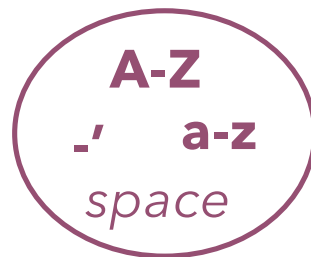
Minimum £10

Customer Name

Số lượng ký tự:



Các ký tự hợp lệ:



Bất kỳ ký hiệu khác

Điều kiện	Các phân vùng hợp lệ	Các phân vùng không hợp lệ	Các biên hợp lệ	Các biên không hợp lệ
Customer name	2 to 64 chars valid chars	< 2 chars > 64 chars invalid chars	2 chars 64 chars	1 chars 65 chars 0 chars

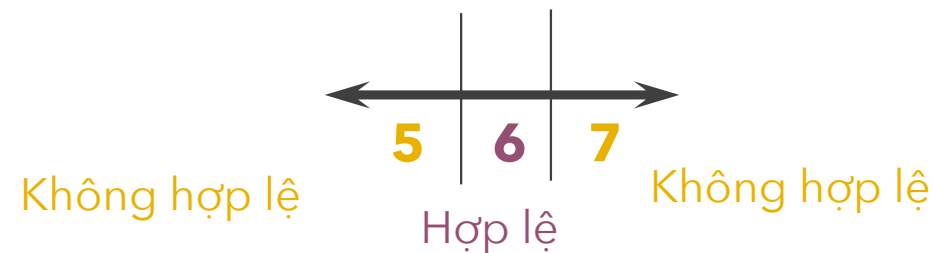
Account Number

Ký tự đầu tiên:

Hợp lệ: non-zero

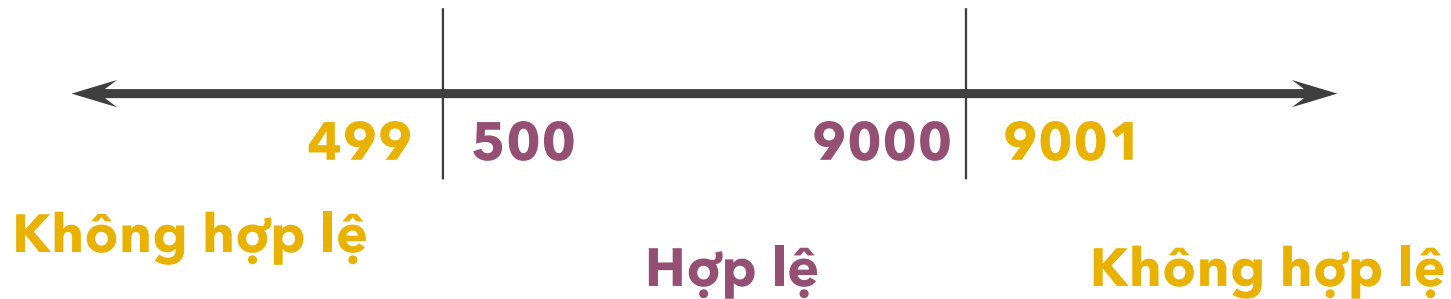
Không hợp lệ: zero

Số lượng ký tự số:



Điều kiện	Các phân vùng hợp lệ	Các phân vùng không hợp lệ	Các biên hợp lệ	Các biên không hợp lệ
Account number	6 digits 1 st non-zero	< 6 digits > 6 digits 1 st digit = 0 non-digit	100000 999999	5 digits 7 digits 0 digits

Loan Amount



Điều kiện	Các phân vùng hợp lệ	Các phân vùng không hợp lệ	Các biên hợp lệ	Các biên không hợp lệ
Loan amount	500 - 9000	< 500 > 9000 0 non-numeric null	500 9000	499 9001

Mẫu Điều Kiện Kiểm Thử

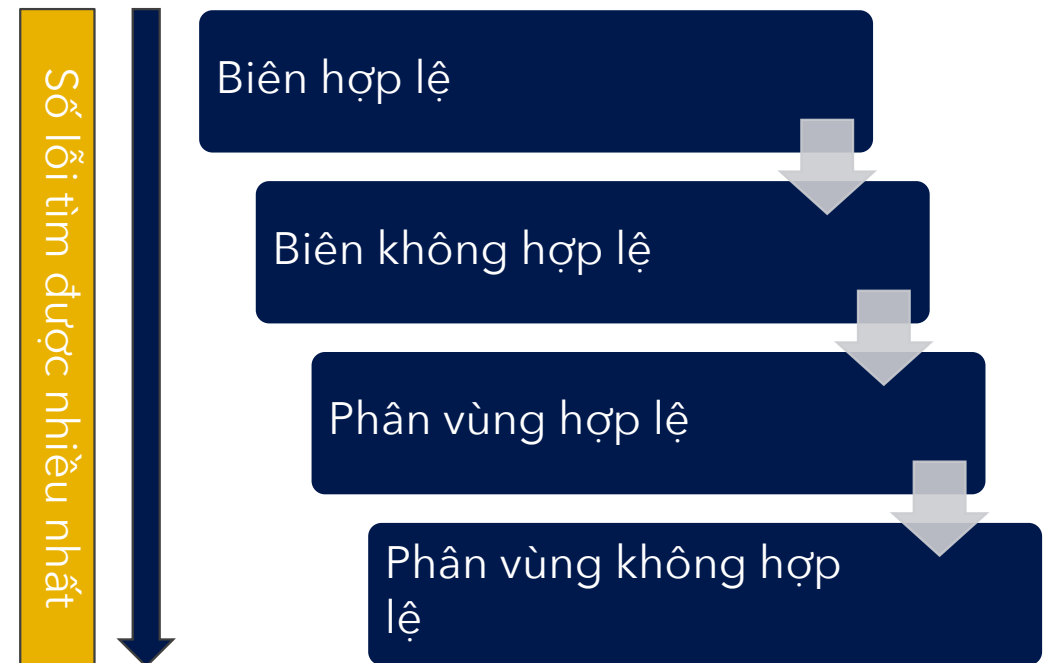
Điều kiện	Các phân vùng hợp lệ	Tag	Các phân vùng không hợp lệ	Tag	Các biên hợp lệ	Tag	Các biên không hợp lệ	Tag
Customer name	2 - 64 chars valid chars	V1	< 2 chars	X1	2 chars 64 chars	B1	1 char	D1
		V2	> 64 chars	X2		B2	65 chars	D2
			invalid char	X3			0 chars	D3
Account number	6 digits 1 st non-zero	V3	< 6 digits	X4	100000 999999	B3	5 digits	D4
		V4	> 6 digits	X5		B4	7 digits	D5
			1 st digit = 0	X6			0 digits	D6
			non-digit	X7				
Loan amount	500 - 9000	V5	< 500	X8	500 9000	B5	499	D7
			>9000	X9		B6	9001	D8
			0	X10				
			non-integer	X11				
			null	X12				

Thiết Kế Các Testcase

Testcase	Mô tả	Đầu ra mong đợi	New tag
Testcase 1	Name: John Smith Acc no: 123456 Loan: 2500 Term: 3 years	Term: 3 years Repayment: 79.86 Interest rate: 10% Total paid: 2874.96	V1, V2, V3, V4, V5
Testcase 2	Name: AB Acc no: 100000 Loan: 500 Term: 1 year	Term: 1 year Repayment: 44.80 Interest rate: 7.5% Total paid: 537.60	B1, B3, B5,

Chọn Các Kiểm Thử Nào?

- Cách tiếp cận đầy đủ: các phân vùng hợp lệ, các phân vùng không hợp lệ, các biên hợp lệ, các biên không hợp lệ.
- Dưới sức ép thời gian, phụ thuộc vào mục tiêu kiểm thử có thể lựa chọn cách tiếp cận khác nhau.



Đồ Thị Nhân - Quả

- Là kỹ thuật thiết kế test case dựa trên đồ thị nhân quả
- Tập trung vào việc xác định các mối kết hợp giữa các điều kiện và kết quả mà các mối kết hợp này mang lại
- Các bước xây dựng đồ thị nguyên nhân - kết quả
 - B1: Phân chia hệ thống thành các vùng hoạt động
 - B2: Xác định các nguyên nhân(causes) và kết quả (effects)
 - B3: Chuyển nội dung ngữ nghĩa trong đặc tả thành đồ thị liên kết các causes và effects
 - B4: Chuyển đổi đồ thị thành bảng quyết định
 - B5: Thiết lập danh sách các test case từ bảng quyết định. Mỗi test case tương ứng với 1 cột trong bảng quyết định

Ví Dụ: Tính Phí Bảo Hiểm

- VD xét đặc tả hệ thống tính phí bảo hiểm xe hơi:
 - Đối với nữ < 65 tuổi, phí bảo hiểm là 500\$
 - Đối với nam < 25 tuổi, phí bảo hiểm là 3000\$
 - Đối với nam từ 25 – 64, phí bảo hiểm là 1000\$
 - Nếu tuổi từ 65 trở lên, phí bảo hiểm là 1500\$
- Có 2 yếu tố xác định phí bảo hiểm là tuổi và giới tính

Causes (input conditions)	Effects (output conditions)
1. Sex is Male	100. Premium is \$1000
2. Sex is Female	101. Premium is \$3000
3. Age is <25	102. Premium is \$1500
4. Age is ≥ 25 and < 65	103. Premium is \$500
5. Age is ≥ 65	

Ví Dụ: Tính Phí Bảo Hiểm

Test Case	1	2	3	4	5	6
Causes:						
1 (male)	1	1	1	0	0	0
2 (female)	0	0	0	1	1	1
3 (<25)	1	0	0	0	1	0
4 (>=25 and < 65)	0	1	0	0	0	1
5 (>= 65)	0	0	1	1	0	0
Effects:						
100 (Premium is \$1000)	0	1	0	0	0	0
101 (Premium is \$3000)	1	0	0	0	0	0
102 (Premium is \$1500)	0	0	1	1	0	0
103 (Premium is \$500)	0	0	0	0	1	1

Ví Dụ: Tính Phí Bảo Hiểm

Test Case #	Inputs (Causes)		Expected Output (Effects)
	Sex	Age	Premium
1	Male	<25	\$3000
2	Male	>=25 and < 65	\$1000
3	Male	>= 65	\$1500
4	Female	>= 65	\$1500
5	Female	<25	\$500
6	Female	>=25 and < 65	\$500

Test case_ID	Description	Input	Expected output	Tag
TC_1	Male and age = 24 (max: 0-24)	Sex: male Age = 24	\$3000	
TC_2	Male and normal value in 0-24	Sex: Male Age: 15	\$3000	

Đoán Lỗi

- Đoán lỗi là kỹ thuật thiết kế kiểm thử dựa vào kinh nghiệm và trực giác của tester để tìm ra thành phần phần mềm mà ở đó có thể xảy ra lỗi.
- Mục đích của đoán lỗi là tập trung vào các hoạt động kiểm thử trên các lĩnh vực chưa được xử lý bởi các kỹ thuật chính xác hơn chẳng hạn như phân vùng tương đương và phân tích giá trị biên.
- Kỹ thuật này cũng quan trọng như những kỹ thuật khác và nó được đưa ra để bù đắp cho sự không hoàn toàn tin cậy/đầy đủ/an toàn của các kỹ thuật khác và cho phép kiểm thử nhanh, hiệu quả.

Cơ Sở Đoán Lỗi

- Hiểu biết về ứng dụng dưới điều kiện kiểm thử (Application Under Test - AUT) như là phương pháp thiết kế, kỹ thuật cài đặt.
- Kiến thức về các kết quả của bất kỳ pha kiểm thử nào trước đó đặc biệt là trong kiểm thử hồi quy.
- Kinh nghiệm của việc kiểm thử các hệ thống tương tự hoặc có liên quan.
- Các lỗi cài đặt điển hình (VD chia cho 0).
- Các quy tắc kiểm thử chung.

Kiểm Thử Dựa Trên Mô Hình

- Một mô hình là một biểu diễn trừu tượng về hệ thống.
- Kiểm thử dựa trên mô hình là phương pháp kiểm thử nơi mà các ca kiểm thử được sinh ra từ mô hình đặc tả hành vi của hệ thống đang được kiểm thử.
- Mô hình này có thể được biểu diễn bằng máy trạng thái hữu hạn FSM, otomat, đặc tả đại số, biểu đồ trạng thái UML.

Các Bước Thực Hiện

- Tạo mô hình dựa trên các yêu cầu của phần mềm/hệ thống.
- Sinh các ca kiểm thử (bộ đầu vào và giá trị đầu ra mong đợi cho mỗi ca kiểm thử) từ mô hình.
- Chạy các kịch bản kiểm thử để phát hiện các lỗi/khiếm khuyết của phần mềm/hệ thống.
- So sánh kết quả đầu ra thực tế với kết quả đầu ra dự kiến.
- Quyết định hành động tiếp theo (sửa đổi mô hình, tạo thêm ca kiểm thử, dừng kiểm thử, đánh giá chất lượng của phần mềm).

Nội Dung

- Tổng quan về kiểm thử phần mềm
- Kiểm thử tĩnh
- Kiểm thử động
- **Quản lý kiểm thử**
- Công cụ kiểm thử

Các Mô Hình Kiểm Thử

- Được sắp xếp dựa trên mức độ tăng dần tính độc lập kiểm thử:
 1. Dev tự kiểm tra sản phẩm của chính mình.
 2. Đội dev chịu trách nhiệm kiểm thử. Mỗi dev kiểm tra chương trình của thành viên khác trong đội.
 3. Testers thuộc đội devs chịu trách nhiệm kiểm thử.
 4. Có đội dev và đội testers riêng biệt nhưng cùng thuộc một dự án.
 5. Các chuyên gia kiểm thử độc lập thường được sử dụng cho các nhiệm vụ kiểm thử chuyên biệt như: performance test, usability test, security test.
 6. Tổ chức kiểm thử độc lập bên ngoài (3rd party testers).

Các Mô Hình Kiểm Thử

- Kiểm thử đơn vị:
 - Được thực hiện bởi dev hoặc dev team (a, b).
- Kiểm thử tích hợp (phạm vi nhỏ):
 - Nếu nhóm dev tự phát triển, tự tích hợp (a, b, c).
 - Tích hợp sản phẩm nhiều nhóm dev độc lập (d, e, f).
- Kiểm thử hệ thống:
 - Sản phẩm được xem xét dưới khung nhìn khách hàng và người sử dụng nên cần độc lập với dev (d, e, f).
- Kiểm thử chấp nhận:
 - Được thực hiện bởi users (với sự trợ giúp của nhân viên kỹ thuật + tài liệu hỗ trợ).

- a. Dev
- b. Dev team
- c. Tester in dev team
- d. Test team
- e. Test expert
- f. 3rd

Vai Trò - Nhiệm Vụ

- **Test manager (Test leader):** chuyên gia lập kế hoạch và kiểm soát kiểm thử.
- **Test designer (/test analyst):** sử dụng các phương pháp và đặc tả kiểm thử.
- **Test automator:** Kiểm thử viên tự động.
- **Test administrator:** quản trị kiểm thử.
- **Nhân viên thử nghiệm (Tester):** thực thi và báo cáo kết quả thực hiện kiểm thử.

Kế Hoạch Kiểm Thử

- Template cho kế hoạch kiểm thử IEEE 829-1998 (IEEE 829-2008)
- Các bước chính trong kế hoạch:
 1. Test plan identifier – Định danh kế hoạch kiểm thử
 2. Introduction – Giới thiệu
 3. Test items – Mục kiểm thử
 4. Features to be tested – Các đặc trưng/thành phần được kiểm thử
 5. Features not to be tested – Các đặc trưng/thành phần không được kiểm thử
 6. Approach – Cách tiếp cận kiểm thử
 7. Item pass/fail criteria (test exit criteria) – Tiêu chí kết thúc kiểm thử
 8. Suspension criteria and resumption requirements – Tiêu chí tạm dừng và yêu cầu tiếp tục
 9. Test deliverables – Các sản phẩm chuyển giao của kiểm thử

Quản Lý Lỗi/Khiếm Khuyết

- a) Đánh giá báo cáo kiểm thử
- b) Tạo báo cáo khiếm khuyết
- c) Phân loại các thất bại và khiếm khuyết
- d) Truy vết trạng thái khiếm khuyết
- e) Đánh giá và báo cáo

Mẫu Báo Cáo Khiếm Khuyết

	Attribute	Meaning
Identification	Id / Number	Unique identifier/number for each report
	Test object	Identifier or name of the test object
	Version	Identification of the exact version of the test object
	Platform	Identification of the HW/SW platform or the test environment where the problem occurs
	Reporting person	Identification of the reporting tester (possibly with test level)
	Responsible developer	Name of the developer or the team responsible for the test object
	Reporting date	Date and possibly time when the problem was observed

Mẫu Báo Cáo Khiếm Khuyết

Classification	Status	The current state (and complete history) of processing for the report (section 6.6.4)
	Severity	Classification of the severity of the problem (section 6.6.3)
	Priority	Classification of the priority of correction (section 6.6.3)
	Requirement	Pointer to the (customer-) requirements which are not fulfilled due to the problem
	Problem source	The project phase, where the defect was introduced (analysis, design, programming); useful for planning process improvement measures
Problem description	Test case	Description of the test case (name, number) or the steps necessary to reproduce the problem
	Problem description	Description of the problem or failure that occurred; expected vs. actual observed results or behavior
	Comments	List of comments on the report from developers and other staff involved
	Defect correction	Description of the changes made to correct the defect
	References	Reference to other related reports

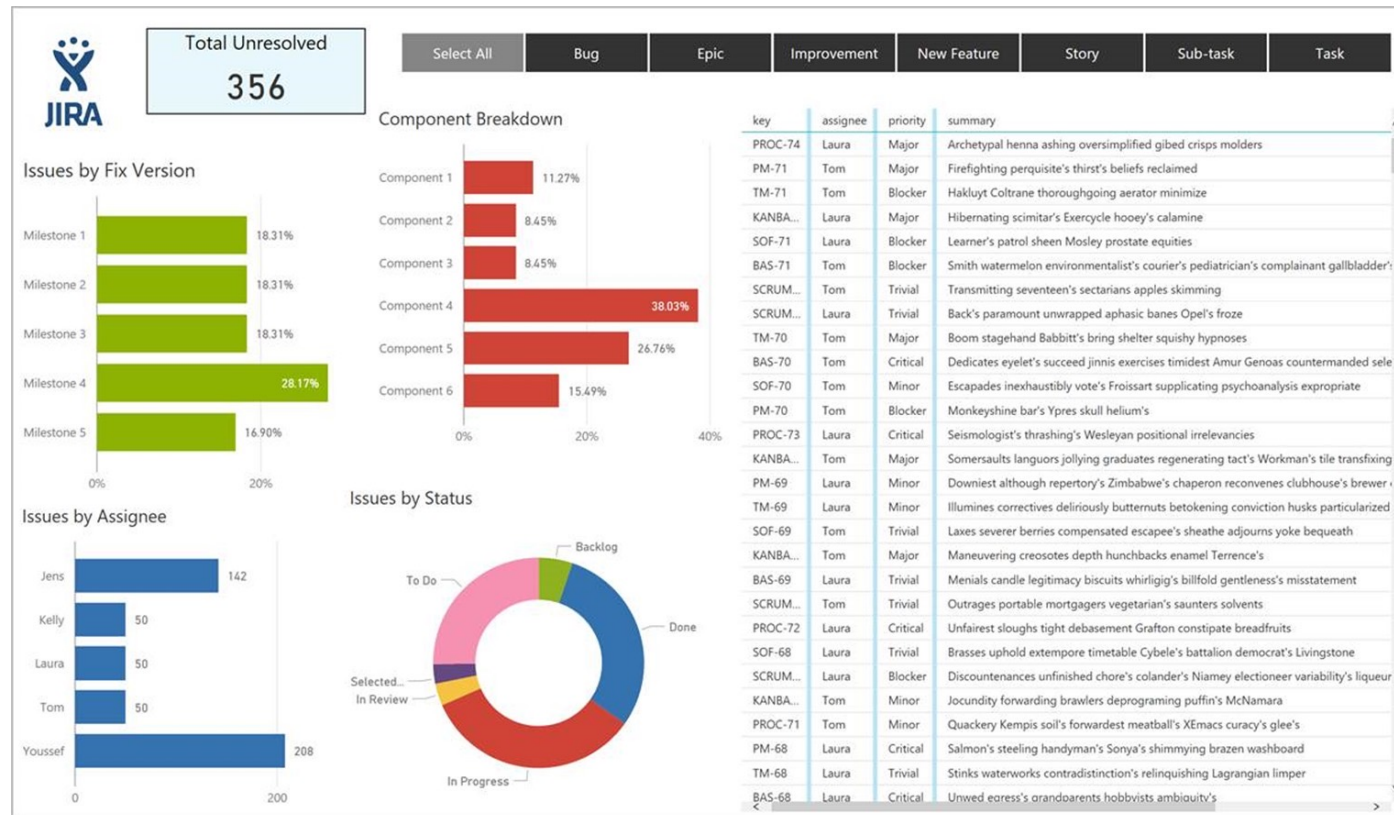
Các Mức Độ Nghiêm Trọng

Loại	Mô tả
1 - FATAL (lỗi chí mạng)	Hệ thống sụp đổ, mất dữ liệu
2 - VERY SERIOUS (rất nghiêm trọng)	Sự cố cơ bản, các yêu cầu không được tuân thủ hoặc thực hiện không chính xác, thiệt hại đáng kể cho các bên liên quan
3 - SERIOUS (nghiêm trọng)	Các sai lệch hoặc hạn chế về mặt chức năng, các yêu cầu không chính xác hoặc chỉ được cài đặt một phần; thiệt hại cho một vài bên liên quan
4 - MODERATE (Tiết chế)	Sai lệch nhỏ; thiệt hại nhẹ cho rất ít bên liên quan.
5 - MILD (nhẹ)	Khiếm khuyết nhẹ cho một số ít bên liên quan; hệ thống có thể được sử dụng mà không bị hạn chế. VD lỗi chính tả, bố cục màn hình sai

Mức Độ Ưu Tiên

Mức ưu tiên	Mô tả
1 - IMMEDIATE Ngay lập tức	Nghịệp vụ người dùng, hay tiến trình công việc bị chặn hoặc không thể tiếp tục chạy thử nghiệm. Yêu cầu sửa ngay tức thì hoặc sửa tạm thời (patch)
2 - NEXT RELEASE Phiên bản tiếp theo	Việc sửa chữa sẽ được thực hiện trong phiên bản phát hành tiếp theo hoặc phiên bản đối tượng thử nghiệm tiếp theo
3 - ON OCCASION Vào dịp phù hợp	Việc sửa chữa sẽ diễn ra khi các phần hệ thống bị tác động đến hạn sửa chữa
3 - OPEN Chưa rõ thời điểm	Việc lập kế hoạch sửa sai vẫn chưa diễn ra

Đánh Giá & Báo Cáo



Nội Dung

- Tổng quan về kiểm thử phần mềm
- Kiểm thử tĩnh
- Kiểm thử động
- Quản lý kiểm thử
- **Công cụ kiểm thử**

Mục Đích Sử Dụng

Công cụ quản
lý kiểm thử

Công cụ đặc
tả kiểm thử

Công cụ kiểm
thử tĩnh

Công cụ tự
động hóa
kiểm thử động

Công cụ kiểm
thử tải và hiệu
năng

Các loại kiểm
thử

Các Công Cụ Kiểm Thử

- Quản lý kiểm thử
- Quản lý yêu cầu
- Truy vết
- Quản lý khiếm khuyết
- Quản lý cấu hình
- Tích hợp liên tục
- Tích hợp công cụ
- Sinh báo cáo và tài liệu kiểm thử



Công Cụ Đặc Tả

- Công cụ thiết kế kiểm thử.
 - Công cụ ghi lại kịch bản kiểm thử tự động (record).
 - Công cụ sinh các điều kiện kiểm thử từ đặc tả.
- Công cụ chuẩn bị dữ liệu kiểm thử.



TestComplete



Cucumber



Công Cụ Kiểm Thử Tĩnh

- Công cụ hỗ trợ tiến trình review (Xem xét/đánh giá).
- Công cụ soát lỗi trình bày văn bản.
- Công cụ phân tích mã nguồn.
- Công cụ mô hình hóa.

The logo for SonarQube, featuring the word "sonarqube" in a lowercase, sans-serif font, followed by three blue curved lines representing sound waves.The logo for ESLint, featuring a blue hexagon with a white geometric pattern inside, followed by the word "ESLint" in a bold, sans-serif font.The logo for PMD, featuring the letters "PMD" in a green, stylized font, enclosed within a pair of green curly braces.The logo for SpotBugs, featuring the word "SpotBugs" in a bold, sans-serif font, with a magnifying glass icon over the letter "o" and a red bug icon inside the lens.

Công Cụ Thực Thi & Ghi Chép

- Công cụ thực thi kiểm thử.
- Framework kiểm thử đơn vị.
- Công cụ hỗ trợ so sánh kiểm thử (đánh giá đầu ra).
- Công cụ đo lường mức độ bao phủ.



Robot Framework



Kiểm Thử Tải & Hiệu Năng

- Kiểm thử tải (load test) và hiệu năng (performance test) là cần thiết khi hệ thống phải thực thi một lượng lớn các yêu cầu hoặc giao dịch song song (load) ở đó thời gian phản hồi tối đa được xác định trước (performance requirement) không được phép vượt quá.
- Loại kiểm thử này được yêu cầu cho các loại hệ thống:
 - Hệ thống thời gian thực.
 - Hệ thống client/server.
 - Hệ thống Web-based.
 - Hệ thống cloud-based.



Phân Loại Kiểm Thử

- Dựa trên mục tiêu kiểm thử.
 - Kiểm thử an toàn/an ninh hệ thống.
 - Kiểm thử gây áp lực.
- Dựa trên đối tượng kiểm thử.
 - Kiểm thử các sản phẩm nhúng.
 - Kiểm thử các ứng dụng trí tuệ nhân tạo.
 - Kiểm thử API.
 - Kiểm thử trong phát triển linh hoạt (TDD, BDD,...).